

Éthique, scope et preuve

Tester seulement ce qui est autorisé, puis rendre la preuve vérifiable.

01

Borner cible, durée et actions

02

Distinguer fait, hypothèse et conclusion

03

Revenir à un état propre

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**Autorisation
explicite avant outil**

Décision observable

02

**Preuve minimale,
horodatée et
expurgée**

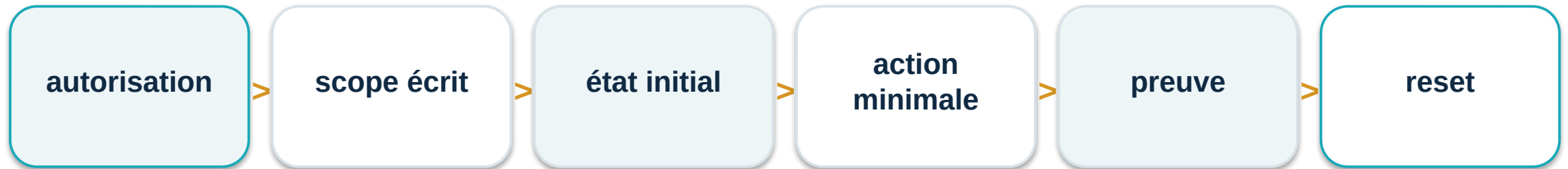
Limite à expliciter

03

**Stop condition en
cas de doute**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Un étudiant reçoit une URL interne sans propriétaire ni fenêtre de test.

FAIT OBSERVÉ

L'URL seule ne prouve ni l'autorisation ni le périmètre.

RISQUE À TESTER

Une action non autorisée expose personnes, données et université.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X Tester d'abord
- X Copier toute la sortie
- X Nettoyer si le temps reste



ÉTAT CORRIGÉ

- OK Demander l'autorisation
- OK Expurger la preuve
- OK Préparer le rollback

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Vérifier le laboratoire avant toute action

TERMINAL LOCAL

```
$ docker compose config -q && docker compose ps
```

SORTIE ATTENDUE

configuration valide; services attendus identifiés, sans cible externe

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Quelles informations manquent avant la première commande ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

Diagnostic + TP01-A: vérifier, démarrer, observer, reset et créer l'index de preuves.

PREUVES À REMETTRE

1 scope signé ou consigne

2 health check réel

3 preuve de cleanup

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

Du DNS à la base

Suivre une requête sans sauter de couche ni confondre protocole et composant.

01

**Ordonner DNS, TCP, TLS
et HTTP**

02

Relier proxy, API et base

03

**Localiser une panne par
couche**

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**Un nom se résout
avant la connexion**

Décision observable

02

**TLS protège le
transport, pas la
logique**

Limite à expliciter

03

**Le proxy ne
remplace pas
l'autorisation API**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Le navigateur affiche une erreur alors que le conteneur API est healthy.

FAIT OBSERVÉ

Un health check local ne couvre pas DNS, TLS, proxy ni parcours utilisateur.

RISQUE À TESTER

Changer plusieurs couches à la fois détruit la valeur diagnostique.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X Redémarrer tout
- X Ajouter -k partout
- X Accuser la base



ÉTAT CORRIGÉ

- OK Tester chaque frontière
- OK Conserver le même path
- OK Comparer status et logs

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Tracer une requête HTTP et le certificat local

TERMINAL LOCAL

```
$ curl -i http://127.0.0.1:8080/api/health && openssl s_client -connect 127.0.0.1:8443  
-servername localhost </dev/null
```

SORTIE ATTENDUE

HTTP 200 JSON; chaîne et identité du certificat local visibles

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Où chercher si TCP fonctionne mais que TLS échoue ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TP01-B + TD01: produire une trace annotée DNS -> DB et une hypothèse falsifiable.

PREUVES À REMETTRE

- 1 requête brute
- 2 certificat observé
- 3 corrélation proxy/API

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

HTTP et contrôles navigateur

Lire requêtes, réponses, cookies et politiques navigateur comme un contrat.

01

**Interpréter méthodes,
statuts et headers**

02

**Distinguer cookie et
session**

03

**Séparer SOP, CORS et
CSP**

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

Le statut décrit un résultat HTTP

Décision observable

02

SameSite n'est pas une autorisation

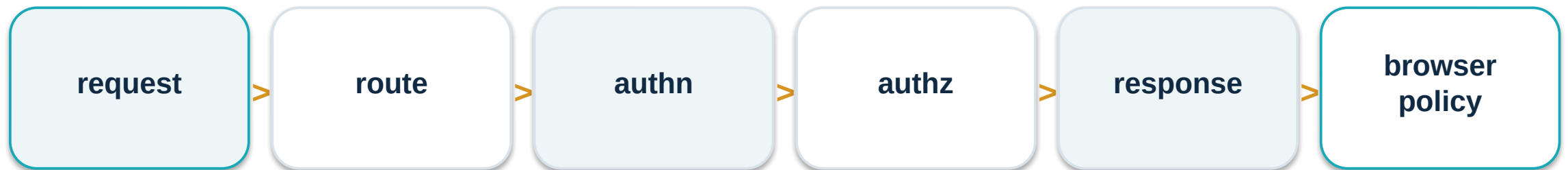
Limite à expliciter

03

CORS contrôle une lecture navigateur, pas curl

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Une API renvoie 200 au bon utilisateur mais le navigateur bloque la lecture cross-origin.

FAIT OBSERVÉ

L'API a traité la requête; la politique navigateur bloque l'accès du script à la réponse.

RISQUE À TESTER

Ouvrir CORS à toutes les origines masque la conception des origines de confiance.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X Access-Control-Allow-Origin: *
- X Cookie sans Secure
- X CSP absente



ÉTAT CORRIGÉ

- OK Origines explicites
- OK Cookie HttpOnly/Secure/SameSite
- OK CSP adaptée aux sources

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Comparer réponse HTTP et décision navigateur

TERMINAL LOCAL

```
$ curl -i -H 'Origin: http://example.invalid' http://127.0.0.1:8080/api/health
```

SORTIE ATTENDUE

status et headers visibles; aucune conclusion automatique sur le comportement d'un navigateur

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Pourquoi curl ne prouve-t-il pas qu'une configuration CORS fonctionne dans le navigateur ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TP02-A: annoter requête/réponse et matrice navigateur/serveur.

PREUVES À REMETTRE

- 1 headers justifiés
- 2 cookie évalué
- 3 limite navigateur/curl

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

Menaces et risque

Passer d'un dessin d'architecture à des scénarios testables et priorisés.

01

Identifier actifs et entrées

02

**Dessiner les frontières
de confiance**

03

**Relier menace, contrôle
et test**

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**Une frontière
marque un
changement de
confiance**

Décision observable

02

**OWASP guide le
raisonnement, pas
le verdict**

Limite à expliciter

03

**Un risque dépend
du contexte**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Le frontend et l'API partagent le même laptop mais pas la même responsabilité.

FAIT OBSERVÉ

Deux processus et interfaces distincts créent une frontière même sur un hôte unique.

RISQUE À TESTER

Un catalogue de vulnérabilités sans actif ni preuve ne priorise rien.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X Lister OWASP
- X Attribuer un score seul
- X Dessiner sans flux



ÉTAT CORRIGÉ

- OK Nommer l'actif
- OK Formuler l'abus
- OK Définir un test falsifiable

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Construire un scénario acteur -> actif -> contrôle

TERMINAL LOCAL

```
$ printf '%s\n' 'acteur=client' 'action=lit objet tiers' 'actif=profil' 'contrôle=owner check' 'test=200/403'
```

SORTIE ATTENDUE

six champs reliés: acteur, action, actif, impact, contrôle et test

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Quelle observation réfuterait votre scénario de BOLA ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TD02 + jalon capstone: DFD, quatre scénarios d'abus et tests associés.

PREUVES À REMETTRE

- 1** DFD légendé
- 2** frontières justifiées
- 3** test négatif/positif

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

Authentification et session

Concevoir un cycle d'identité qui résiste avant, pendant et après la connexion.

01

Expliquer hash, MFA et login

02

Suivre création, rotation et révocation

03

Comparer cookie de session et JWT

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

Le hash protège au repos

Décision observable

02

MFA réduit un risque, sans tout résoudre

Limite à expliciter

03

Logout doit invalider le contexte serveur

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Un identifiant de session reste inchangé avant et après authentification.

FAIT OBSERVÉ

La continuité de l'identifiant rend plausible une fixation de session.

RISQUE À TESTER

Ajouter une expiration longue ne corrige pas l'absence de rotation.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X Stockage réversible
- X Token éternel
- X Logout côté écran



ÉTAT CORRIGÉ

- OK Hash adapté
- OK Rotation à l'authn
- OK Révocation et délai borné

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Observer les changements du contexte d'authentification

TERMINAL LOCAL

```
$ curl -i -c /tmp/session.cookies -b /tmp/session.cookies  
http://127.0.0.1:8080/api/health
```

SORTIE ATTENDUE

headers Set-Cookie/Cookie observables ou absence explicitement documentée

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Que doit invalider un logout fiable ?

1 · Position

Formulez votre réponse

2 · Preuve

**Quelle observation la
soutient ?**

3 · Limite

**Qu'est-ce qui reste non
prouvé ?**

Passage à la pratique supervisée

TP02-B: observer login, expiration, rotation et logout; proposer deux tests négatifs.

PREUVES À REMETTRE

- 1 avant/après login
- 2 rejet après logout
- 3 aucun secret dans la preuve

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

Autorisation et BOLA

Décider côté serveur qui peut faire quoi sur quel objet, à chaque requête.

01

Distinguer authn et authz

02

**Comparer RBAC et
décision objet**

03

Prouver 200/401/403

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**Le rôle ne prouve
pas la propriété**

Décision observable

02

**Le client contrôle
l'identifiant
demandé**

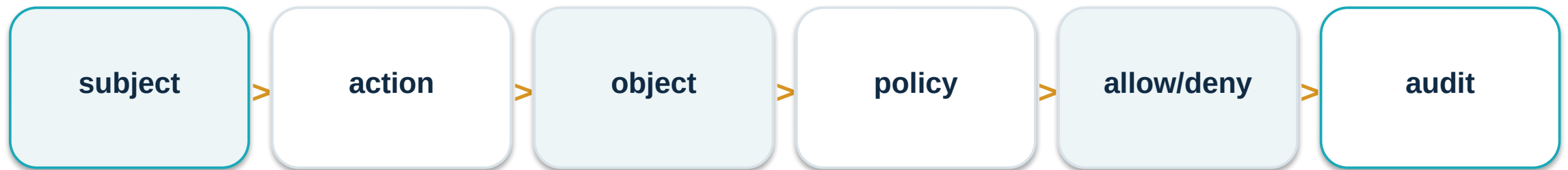
Limite à expliciter

03

**Chaque objet exige
une décision
serveur**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Alice change /users/1 en /users/2 et reçoit le profil de Bob.

FAIT OBSERVÉ

L'identité est valide, mais l'API n'a pas vérifié la propriété de l'objet.

RISQUE À TESTER

Cacher le bouton ou utiliser un UUID ne remplace pas le contrôle serveur.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X Contrôle frontend
- X Rôle user seulement
- X Objet prévisible



ÉTAT CORRIGÉ

- OK Owner/admin côté API
- OK Deny par défaut
- OK Journal sans donnée sensible

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Établir la matrice 200/401/403

TERMINAL LOCAL

```
$ curl -sS -o /dev/null -w '%{http_code}'  
' -H 'Authorization: Bearer TOKEN' http://127.0.0.1:8080/api/users/2
```

SORTIE ATTENDUE

200 en état vulnérable; 403 après correction; 401 sans identité; 200 pour l'objet propre

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Pourquoi faut-il retester aussi l'objet d'Alice après la correction ?

1 · Position

Formulez votre réponse

2 · Preuve

**Quelle observation la
soutient ?**

3 · Limite

**Qu'est-ce qui reste non
prouvé ?**

Passage à la pratique supervisée

TP03: reproduire BOLA, modifier le contrôle, obtenir 200/401/403 et documenter le risque résiduel.

PREUVES À REMETTRE

1 baseline propriétaire

2 refus objet tiers

3 log authz expurgé

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

Validation et SQLi

Séparer strictement structure SQL et données, puis prouver la non-régression.

01

**Tracer source, validation
et sink**

02

**Expliquer la requête
paramétrée**

03

**Retester attaque et usage
légitime**

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

Le typage réduit les entrées possibles

Décision observable

02

Les paramètres séparent code et donnée

Limite à expliciter

03

Un filtre d'apostrophes est contournable

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Une recherche concatène q dans SQL et un payload renvoie tout le catalogue.

FAIT OBSERVÉ

La variation du nombre de lignes établit un comportement injectable dans le lab.

RISQUE À TESTER

Supprimer certains caractères altère les données et laisse d'autres syntaxes.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X Concaténation
- X Blacklist
- X Compte DB propriétaire



ÉTAT CORRIGÉ

- OK Paramètres driver
- OK Validation métier
- OK Rôle DB minimal

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Comparer baseline, payload et requête paramétrée

TERMINAL LOCAL

```
$ curl -sS --get --data-urlencode "q=' OR '1'='1" http://127.0.0.1:8080/api/products
```

SORTIE ATTENDUE

plusieurs lignes avant; aucune ligne après; q=lamp reste fonctionnel

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Pourquoi la validation seule ne remplace-t-elle pas les paramètres SQL ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TP04-A: SQLi locale, correction source, test négatif et recherche légitime.

PREUVES À REMETTRE

- 1 diff du sink
- 2 résultat avant/après
- 3 privilèges DB relevés

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

XSS, CSRF et CSP

Adapter le contrôle au contexte de sortie et au modèle d'envoi des credentials.

01

Distinguer validation et encodage

02

Expliquer CSRF et SameSite

03

Utiliser CSP en défense supplémentaire

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**L'encodage dépend
du contexte
HTML/JS/URL**

Décision observable

02

**CSRF exploite
l'envoi automatique**

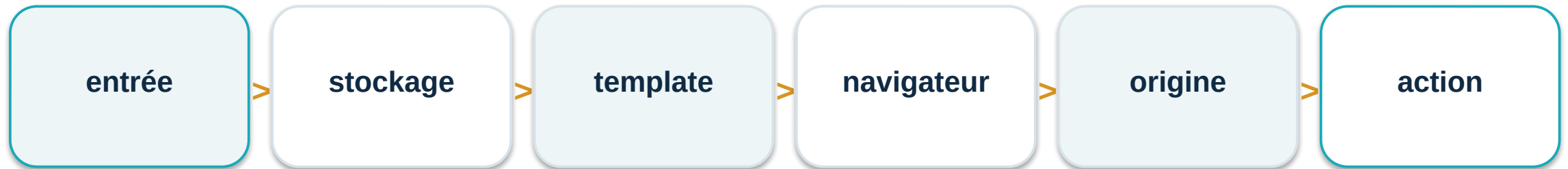
Limite à expliciter

03

**CSP ne répare pas
un sink dangereux**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Un commentaire est inséré dans `innerHTML`; le cookie est envoyé automatiquement.

FAIT OBSERVÉ

Le sink interprète du HTML contrôlé et l'action serveur peut manquer de preuve d'intention.

RISQUE À TESTER

Une CSP permissive ou SameSite seul laisse des chemins d'attaque.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X innerHTML
- X CSRF sans token
- X CSP unsafe-inline



ÉTAT CORRIGÉ

- OK.textContent/encoding
- OK.token + origine
- OK.CSP restrictive mesurée

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Comparer texte sûr et interprétation HTML

TERMINAL LOCAL

```
$ printf '%s\n' '<img src=x onerror=alert(1)>' | python3 -c 'import html,sys; print(html.escape(sys.stdin.read()))'
```

SORTIE ATTENDUE

les caractères actifs deviennent des entités; le contexte exact reste à vérifier

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Pourquoi SameSite=Lax n'est-il pas une correction universelle de CSRF ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TP04-B + Quiz 2: cas XSS/CSRF locaux, correction, CSP et retest.

PREUVES À REMETTRE

- 1 sink identifié
- 2 requête CSRF comparée
- 3 CSP expliquée sans sur-promesse

STOP CONDITION

Cible inattendue
Secret réel
Reset incertain

Commande, fichiers et SSRF

Réduire l'interprétation implicite et borner chaque accès système ou réseau.

01

Éviter le shell pour les commandes

02

Canonicaliser et isoler les fichiers

03

Appliquer allowlist et egress au SSRF

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**Une extension ne
prouve pas le
contenu**

Décision observable

02

**Un chemin final se
vérifie après
résolution**

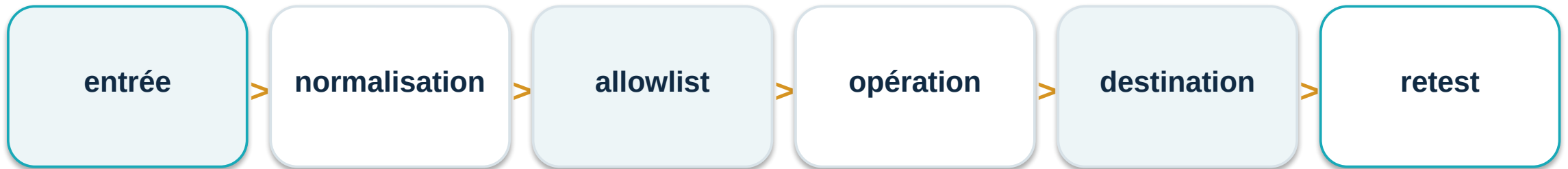
Limite à expliciter

03

**SSRF combine
validation et
segmentation**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Un nom `../private/demo.txt` sort du répertoire public; une URL vise `db:5432`.

FAIT OBSERVÉ

Les entrées franchissent une frontière fichier ou réseau non prévue.

RISQUE À TESTER

Une blacklist de `../` ou de `localhost` ne couvre ni encodages ni redirections.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X shell=True
- X nom de fichier brut
- X URL arbitraire



ÉTAT CORRIGÉ

- OK API sans shell
- OK nom généré + stockage hors webroot
- OK allowlist + egress

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Observer une traversal bornée puis le refus

TERMINAL LOCAL

```
$ curl -sS --get --data-urlencode 'name=../sample-private/demo.txt'  
http://127.0.0.1:8080/api/file
```

SORTIE ATTENDUE

chaîne synthétique avant; refus contrôlé après; fichier public toujours accessible

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

À quel moment comparer le chemin à la racine autorisée ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

**TP05 + jalon capstone: cas bornés, corrections et retests
commande/traversal/upload/SSRF.**

PREUVES À REMETTRE

- 1** aucun fetch externe
- 2** fichier synthétique
- 3** cas légitime préservé

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

Sécurité des API

Faire respecter le contrat API sur identité, objet, propriétés, volume et erreurs.

01

Inventorier routes et schémas

02

Bloquer BOLA et mass assignment

03

Borner quotas et données exposées

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**OpenAPI décrit
l'intention, pas toute
la réalité**

Décision observable

02

**Une allowlist de
propriétés bloque
l'over-posting**

Limite à expliciter

03

**429 exige une
fenêtre explicite**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Le client ajoute `role=admin` à un JSON accepté directement par le modèle persistant.

FAIT OBSERVÉ

Une propriété non prévue franchit le binding client -> objet serveur.

RISQUE À TESTER

Masquer le champ dans l'interface ne change pas le JSON envoyé.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X bind automatique
- X données excessives
- X quota global flou



ÉTAT CORRIGÉ

- OK DTO allowlist
- OK réponse minimale
- OK quota par identité + 429

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Comparer OpenAPI et propriétés réellement acceptées

TERMINAL LOCAL

```
$ curl -sS http://127.0.0.1:8080/api/openapi.json | python3 -m json.tool
```

SORTIE ATTENDUE

routes et schémas déclarés; les tests négatifs révèlent les écarts d'implémentation

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Que manque-t-il à un inventaire OpenAPI pour conclure sur la sécurité ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TP06-A: inventaire, BOLA, mass assignment, rate limiting et logs.

PREUVES À REMETTRE

- 1** route/identité/objet
- 2** propriété rejetée
- 3** 429 et fenêtre documentés

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

TLS, proxy et conteneurs

Durcir la frontière d'entrée sans casser la visibilité ni faire confiance aux mauvais headers.

01

Expliquer certificat et terminaison TLS

02

Valider forwarded headers

03

Réduire privilèges conteneur

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**Le proxy est une
frontière de
confiance**

Décision observable

02

**HSTS suppose
HTTPS fiable**

Limite à expliciter

03

**Non-root et read-
only limitent
l'impact**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

L'API croit X-Forwarded-For envoyé directement par un client non fiable.

FAIT OBSERVÉ

Le header ne devient fiable que s'il vient d'un proxy connu qui remplace la valeur entrante.

RISQUE À TESTER

Faire confiance à tous les proxies permet d'inventer origine ou schéma.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X TLS -k permanent
- X trust proxy=*
- X root + filesystem writable



ÉTAT CORRIGÉ

- OK CA locale documentée
- OK proxy connu + overwrite
- OK non-root/read-only/cap_drop

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Vérifier certificat et headers de défense

TERMINAL LOCAL

```
$ openssl s_client -connect 127.0.0.1:8443 -servername localhost </dev/null  
2>/dev/null | openssl x509 -noout -subject -issuer -dates
```

SORTIE ATTENDUE

subject, issuer et validité visibles; aucun jugement sans hostname et chaîne

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Pourquoi $\text{curl} \mathbf{L} - \mathbf{k}$ est-il utile au diagnostic mais pas une remédiation ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TP06-B: durcir Nginx/TLS/conteneurs et vérifier sans exposition réseau.

PREUVES À REMETTRE

- 1 **certificat expliqué**
- 2 **headers source -> proxy**
- 3 **capabilities relevées**

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

Sécurité PostgreSQL

Limiter les droits, protéger les données et prouver qu'une restauration fonctionne.

01

**Séparer propriétaire,
migration et runtime**

02

**Expliquer chiffrement
transit/repos**

03

**Tester backup, restore et
audit**

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**Le compte runtime
n'est pas
propriétaire**

Décision observable

02

**Un backup non
restauré reste une
hypothèse**

Limite à expliciter

03

**L'audit exclut
secrets et données
inutiles**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

L'application se connecte avec le propriétaire de la base et peut modifier le schéma.

FAIT OBSERVÉ

Une compromission applicative hérite de privilèges dépassant les besoins runtime.

RISQUE À TESTER

Un mot de passe fort ne compense pas des grants excessifs.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X rôle unique owner
- X backup jamais testé
- X logs SQL sensibles



ÉTAT CORRIGÉ

- OK rôles séparés
- OK restore isolé vérifié
- OK audit minimal et protégé

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Lire les privilèges effectifs du rôle runtime

TERMINAL LOCAL

```
$ psql -Atc "select current_user, current_database();" && psql -c '\dp'
```

SORTIE ATTENDUE

rôle et grants observables; CREATE/ALTER absents pour le runtime attendu

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Quelle preuve transforme un backup en capacité de restauration ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TP07-A + jalon capstone: rôles minimaux, backup/restore et audit expurgé.

PREUVES À REMETTRE

- 1** grants avant/après
- 2** restore prouvé
- 3** événement audit sans secret

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

DevSecOps et exceptions

Transformer les scans en décisions reproductibles, explicables et temporaires.

01

**Distinguer SAST, SCA,
DAST et secrets**

02

**Trier exploitabilité et
impact**

03

**Versionner une exception
expirante**

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**Un scanner produit
un signal**

Décision observable

02

**Une panne d'outil
n'est pas un PASS**

Limite à expliciter

03

**Chaque exception a
owner, raison et
échéance**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Une CVE high est non atteignable; le pipeline l'ignore sans justification ni date.

FAIT OBSERVÉ

L'absence de trace rend la décision impossible à réexaminer après changement.

RISQUE À TESTER

Désactiver le gate transforme une exception locale en dette invisible.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X allow failure
- X suppression globale
- X scan non épinglé



ÉTAT CORRIGÉ

- OK triage documenté
- OK exception ciblée et expirante
- OK outil/version enregistrés

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Exécuter tests, SAST et audit de dépendances

TERMINAL LOCAL

```
$ python -m pytest -q && bandit -q -r app && pip-audit -r requirements.txt
```

SORTIE ATTENDUE

chaque commande retourne un code exploité; finding ou panne ne sont jamais convertis silencieusement en PASS

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Quelles métadonnées minimales exige une exception ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TP07-B + checkpoint: pipeline local, triage et rapport d'exception versionné.

PREUVES À REMETTRE

- 1** codes de sortie
- 2** triage signé
- 3** exception avec échéance

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**

Détection, incident et rapport

Relier un signal à des faits, une décision proportionnée, une correction et un retest.

01

**Corréler logs, métriques
et traces**

02

**Construire une timeline
sans sur-attribuer**

03

**Communiquer risque et
incertitude**

Palette pédagogique provisoire · charte officielle À confirmer

Trois repères avant l'outil

01

**Un signal n'est pas
une attribution**

Décision observable

02

**La corrélation exige
temps et identifiant**

Limite à expliciter

03

**Le rapport sépare
faits, hypothèses et
risque résiduel**

Preuve à conserver

Le flux et ses décisions



À chaque flèche : qui fait confiance à qui ? quelle donnée ? quel contrôle ? quelle preuve ?

Cas guidé : faits avant verdict

Les 401 augmentent après une mise en production; aucune trace ne relie requête et changement.

FAIT OBSERVÉ

La métrique établit une variation, pas sa cause ni un attaquant.

RISQUE À TESTER

Bloquer une plage réseau sans contexte peut aggraver l'incident.

Règle : une hypothèse utile indique l'observation qui pourrait la réfuter.

Vulnérable -> corrigé -> vérifié

ÉTAT VULNÉRABLE

- X alerte=incident
- X loguer Authorization
- X corriger sans baseline



ÉTAT CORRIGÉ

- OK corrélation_id
- OK données minimales
- OK retest + risque résiduel

Même test négatif + parcours légitime positif + risque résiduel explicite

Démo : Interroger un signal et chercher sa corrélation

TERMINAL LOCAL

```
$ curl -sS 'http://127.0.0.1:9090/api/v1/query?query=websec_http_requests_total' |  
python3 -m json.tool
```

SORTIE ATTENDUE

status success et séries; une timeline exige ensuite logs/traces et contexte de changement

Prédire -> exécuter -> lire -> conclure -> retester -> reset

Checkpoint de compréhension

Qu'est-ce qui permet de passer d'un signal à une décision ?

1 · Position

Formulez votre réponse

2 · Preuve

Quelle observation la soutient ?

3 · Limite

Qu'est-ce qui reste non prouvé ?

Passage à la pratique supervisée

TD04 + capstone: triage, remédiation, retest, rapport et défense individuelle.

PREUVES À REMETTRE

- 1** timeline sourcée
- 2** décision proportionnée
- 3** rapport multi-audience

STOP CONDITION

**Cible inattendue
Secret réel
Reset incertain**